Async credit processing service

Approach and request flow

I kept the API's job small: accept a credit, save it, and queue the work. FastAPI handles requests, RabbitMQ holds tasks, and Celery runs the worker. MongoDB stores balances and tracks each credit.

The main concern was retries. If a task runs twice, the user should still receive the money only once.

What happens to a request

1. The client sends `POST /credits` with a user ID and amount. The API validates it, saves a credit record, and sets the user to `processing`.
2. The API attempts to queue the credit ID and returns **202 Accepted**, with an ID for checking progress. It does not wait for the balance update.
3. The worker checks the credit. If it is pending, it adds the amount and marks the credit `processed` in the same MongoDB transaction.
4. GET endpoints let the client check a credit or a user's balance. When credits overlap, the user stays `processing` until the pending work finishes.

Why keep a separate credit record

The balance alone cannot tell the worker whether it has already handled a task. A credit record can. Updating both together also handles a crash after the database commits: the next attempt sees that the credit is complete and leaves the balance alone.

The record also holds the client's `Idempotency-Key`. Reusing a key returns the original credit; changing the amount or user returns a conflict. The header is optional, but clients need it to retry requests safely.

Assumptions

- One currency, positive amounts, and no fractions smaller than a cent. `Decimal` and `Decimal128` preserve money values.
- Unknown users are created automatically. The worker creates a missing balance field on the first credit.
- A 202 response means the credit was saved, not that the balance has changed.

Recovery testing and trade-offs

What if the queue is unavailable

Saving to MongoDB and publishing to RabbitMQ are separate steps. A crash between them could otherwise leave a saved credit with no task in the queue.

I added a dispatcher that finds pending credits and republishes them after a 30-second lease expires. It can publish duplicates, so the worker still needs its transaction check. MongoDB holds the actual outcome; a Celery task ID does not prevent duplicate credits.

Connection and temporary transaction failures retry. Non-transient database errors are marked `failed` if the database can save that outcome. Unexpected errors stay pending and are logged. A CLI command lets an operator requeue failures after fixing the cause.

Tests I focused on

Beyond validation and the normal request flow, I focused on cases that could produce the wrong balance:

- Running the same credit repeatedly, including concurrent attempts.
- Applying different credits to one user without losing an update.
- Failing between the balance update and completion record, then checking rollback.
- Recovering failed publication and keeping status correct when credits overlap.

All **34 tests passed**, including an end-to-end test with real MongoDB, RabbitMQ, and Celery. Worker and broker stop-and-restart checks also passed. Replaying a committed credit was tested; killing a worker at the exact commit boundary was not.

Running it and what I would change next

Docker Compose starts the services together so reviewers do not need to configure each dependency. MongoDB uses a single-node replica set for transactions. The README includes startup, sample requests, and test commands.

The dispatcher is the main trade-off: recovery is simple, but waiting tasks can get published again. With more traffic, I would measure queue age and duplicate dispatches before adjusting that policy and adding admission limits.

Before using this for real account balances, I would add account authorization, managed secrets, tested backups, and reconciliation. I would also test database failover and sustained load. Those are outside this local exercise's scope.